# Literature Review: Pre-Publication Ambiguity Detection in Internet-Drafts

## Review Scope and Method

Internet standards are unusual software-engineering artefacts: they are written primarily in natural language, contain normative requirements alongside formal fragments, and evolve through multiple Internet-Draft revisions before publication as RFCs. This review positions the proposed project against four bodies of work: ambiguity detection in requirements and protocol specifications; automated interpretation and formalization of RFCs; RFC-guided implementation and compliance analysis; and empirical or NLP-based studies of the IETF revision process.

The review adopts a targeted narrative approach, drawing primarily on the project evidence base and incorporating methodological studies on multi-version document alignment and edit-intent analysis to strengthen the revision-aware component. The literature selection focuses on studies that directly inform ambiguity detection, RFC analysis, document evolution, evidence grounding, and evaluation methodology.

## 1 Ambiguity in Natural-Language Specifications

RFCs use normative terms such as MUST, SHOULD, and MAY to communicate requirement strength [1], while RFC 8174 clarifies when capitalization gives these terms their BCP 14 meaning [2]. This vocabulary improves consistency but does not make a requirement unambiguous by itself. A clause may still leave a term undefined, omit a boundary case, conflict with another section, rely on an incorrect reference, or admit several operational interpretations.

Requirements-engineering research provides a useful conceptual foundation. Berry, Kamsties, and Krieger catalogue linguistic sources of ambiguity and distinguish ambiguity from general poor writing [3]. Yang and colleagues study nocuous and anaphoric ambiguity, emphasizing that ambiguity is harmful when different readers can form materially different interpretations [4, 5]. This distinction is important for protocol standards because the objective is not stylistic simplification; it is to reduce divergent implementations and interoperability failures.

Automated work spans handcrafted rules, supervised learning, contextual models, and LLMs. Ezzini et al. compare approaches to anaphoric ambiguity [6]; Veizaga et al. show the value of interpretable requirement-smell signals [7]; Izhar et al. combine engineered information with machine learning [8]; and Bashir et al. investigate LLM-based ambiguity detection and explanation in industrial requirements [9]. Zhao et al. show that NLP support for requirements engineering is broad but still constrained by dataset quality, generalization, and limited industrial validation [10]. These methods motivate vague-term, anaphora, reference, classical-ML, transformer, and LLM components, but general requirements datasets do not reproduce the dense cross-references, domain terminology, ABNF, tables, diagrams, and inter-document dependencies of Internet-Drafts.

## 2 Protocol-Specific Ambiguity Detection

SAGE is an important precursor because it uses semantic parsing and interaction with protocol authors to expose ambiguous or under-specified specification text before code generation [11]. It demonstrates that ambiguity can be identified through an operational task and that author clarification provides strong evidence that a reported issue is meaningful. However, SAGE targets components suitable for semantic parsing and code generation, relies on a manually designed grammar and author-in-the-loop workflow, and does not provide a revision-aware benchmark for scalable screening of heterogeneous Internet-Drafts.

RFCScope is the closest direct baseline [12]. It derives a taxonomy of logical ambiguity from verified technical errata in Standards Track RFCs, covering direct and indirect inconsistencies, conflicts with accepted domain knowledge, undefined terms, incomplete constraints, indirect under-specification, and incorrect or missing references. Its LLM-centred pipeline constructs relevant cross-document context,

applies subtype-aware analysis, and uses a separate evaluator to reject unsupported findings. The reported results show that LLMs can identify consequential logical issues that simple lexical rules may miss.

RFCScope also exposes limitations central to this project: its emphasis on precision may leave false negatives; its taxonomy is derived from observed errata; context construction can select noisy or incorrect evidence; evolving proprietary models affect reproducibility; and manual inspection remains necessary. Most importantly, RFCScope analyses already published RFCs. It does not test whether the same issue could have been detected from an earlier Internet-Draft using only information available at that time. The proposed work therefore treats RFCScope as the closest taxonomy and detector baseline while preserving an independent pre-publication evaluation boundary. Verified errata are candidate evidence, not automatic ambiguity labels, and require clause mapping and adjudication before use as gold positives.

## 3 Automated RFC Interpretation and Compliance

A related research stream converts RFC prose into structured or operational representations. PROSPER uses an LLM to extract finite-state machines from RFCs and reports benefits from combining text with diagram artefacts [13]. Long et al. extend RFC interpretation to protocol code generation, showing progress in compilation and interoperability while still reporting difficulties with ambiguous content, low-level logic, and generalization [14]. These studies motivate resolved terminology, cross-reference retrieval, evidence spans, and structured intermediate representations, but their extraction or execution metrics are not substitutes for clause-level ambiguity detection.

Implementation-oriented work further demonstrates that evolving specification text has practical consequences. EBugDec models requirement changes across published RFCs and checks whether protocol implementations incorporate primitive and derivative rules, showing that standards evolution can produce real implementation inconsistencies [15]. RFCAUDIT uses semantic indexing and demand-driven LLM agents to compare implementations with RFC requirements [16]. These systems support the practical motivation for early ambiguity detection and the need for traceable evidence, while remaining different tasks: an implementation may violate a clear requirement, and implementation divergence does not by itself prove that the specification is ambiguous.

## 4 IETF Revision History, Clause Alignment, and Edit Intent

Empirical work on the IETF explains why document history is a meaningful research object. McQuistin et al. analyse large-scale RFC, email, and authorship data and frame standardization as a long-running socio-technical process rather than a single act of publication [17]. Bian and Welzl address the textual workflow directly by retrieving IETF discussions related to Internet-Draft content and using LLMs to support document editing [18]. Their work shows that discussion and revision context can help explain why a requirement changed, which is useful for label adjudication.

The revision-alignment problem also has direct methodological precedents outside the RFC domain. ARXIVEDITS provides gold sentence alignment across multiple versions of 751 full papers and models insertion, deletion, rephrasing, splitting, merging, reordering, fine-grained edits, and edit intentions [19]. CASIMIR scales multi-version analysis to 15,646 scientific articles, aligning consecutive versions at sentence level while retaining paragraph location and associating revised sentence pairs with edit-intent information [20]. These studies do not address Internet-Draft ambiguity, but they demonstrate that version alignment and edit-intent classification should be treated as distinct tasks rather than reduced to a raw text diff.

This distinction is essential for the proposed benchmark. A textual difference between two Internet-Draft revisions should not automatically become an ambiguity label because revisions may reflect editorial improvement, new functionality, terminology updates, reference maintenance, or technical redesign. Revision evidence should therefore support, but not replace, manual or expert adjudication of why a clause changed.

## 5 Synthesis and Research Gap

The literature establishes five findings. First, harmful ambiguity is semantic and context-dependent, so surface-level smells are useful candidate signals but insufficient final evidence. Second, protocol specifications require domain-aware interpretation across sections, documents, diagrams, and formal fragments. Third, LLMs can recover complex protocol logic but introduce context-selection, hallucination, model-drift, and cost concerns. Fourth, RFC-derived rules reveal real implementation defects, confirming the practical importance of specification quality. Fifth, the IETF drafting process contains revision and discussion evidence that current ambiguity detectors largely do not exploit.

Within the literature reviewed, no study was identified that combines all of the following:

- detection at the Internet-Draft stage using only contemporaneously available information;
- clause alignment across successive draft revisions and the final RFC;
- labels that distinguish ambiguity clarification from unrelated document change;
- a controlled comparison of heuristic, classical ML, transformer, LLM-only, and hybrid methods;
- document-family and temporal evaluation that prevents near-duplicate revision leakage;
- joint evaluation of detection quality, calibration, evidence faithfulness, cost, latency, and review workload.

This gap defines the project contribution. The novelty is not simply applying another LLM to RFC text. It is constructing a revision-aware, provenance-preserving benchmark and using it to test when contemporaneous revision signals improve detection, when evidence-grounded LLM verification is justified, and whether a calibrated hybrid can retain high recall while reducing LLM calls and reviewer workload.

## 6 Methodological Implications for the Proposed Benchmark

The literature suggests that the primary unit should retain the target clause, document family, revision, section, aligned earlier or later clauses, and label evidence. Gold, silver, and uncertain labels should remain separate from revision-reason labels such as ambiguity clarification, editorial change, new functionality, terminology change, reference correction, or technical redesign. Future information may be used to adjudicate historical outcomes, but prediction-time features for revision `-NN` must be restricted to information that existed at `-NN`. Evaluation should therefore prioritize document-family and temporal holdouts over random clause splitting, while using PR-oriented detection metrics, calibration, evidence correctness, latency, cost, and the proportion of clauses routed to LLM or human review. These choices directly support the project research questions without turning the literature review into a full methodology chapter.

# 7 Comparative Summary

| Study | Primary task | Main relevance | Boundary relative to this project |
|---|---|---|---|
| SAGE [11] | Protocol disambiguation / code generation | Pre-publication, author-grounded disambiguation | Workflow- and grammar-specific; no revision-aware benchmark |
| RFCScope [12] | Logical ambiguity detection | Closest taxonomy and detector baseline | Published RFC focus; context/model reproducibility limits |
| PROSPER [13] | FSM extraction from RFCs | Structured interpretation and multimodal context | Extraction task, not ambiguity classification |
| EBugDec [15] | RFC evolution / implementation bugs | Shows standards evolution has operational consequences | Published RFC evolution, not draft-revision ambiguity labels |
| RFCAUDIT [16] | RFC-code conformance auditing | Demand-driven retrieval and LLM reasoning | Implementation-level target; recent preprint |
| Bian & Welzl [18] | IETF discussion retrieval / assisted editing | Connects draft text with feedback and revision context | Does not create ambiguity labels or compare detectors |
| ARXIVEDITS / CASIMIR [19, 20] | Revision alignment and edit intent | Methods for multi-version alignment and change-reason analysis | Scientific-writing domain rather than Internet-Drafts |

# 8 Conclusion

Existing work demonstrates that ambiguity in protocol specifications is real, consequential, and technically difficult to detect. Requirements-engineering methods offer interpretable signals and supervised baselines; SAGE demonstrates author-grounded disambiguation; and RFCScope provides the strongest protocol-specific taxonomy and published-RFC detector. RFC interpretation and compliance studies further show that specification wording has measurable effects on implementation quality, while revision-alignment research provides methods for tracking how text and edit intent evolve across versions.

The unresolved question is whether consequential ambiguity can be detected reliably before publication and whether contemporaneous revision history provides useful evidence without leaking future information. A benchmark built from aligned Internet-Draft families, adjudicated revision outcomes, mapped errata, and fixed family/time splits would address this gap. Comparing heuristic, classical ML, transformer, LLM-only, and hybrid approaches under the same evaluation protocol would additionally determine when LLM reasoning improves results enough to justify its cost and instability. This combination of pre-publication scope, revision-aware evidence, leakage control, and cost-aware hybrid evaluation is the central research contribution of the proposed project.

# References


[1] S. Bradner, "Key words for use in RFCs to indicate requirement levels," RFC 2119, 1997. doi: 10.17487/RFC2119.

[2] B. Leiba, "Ambiguity of uppercase vs lowercase in RFC 2119 key words," RFC 8174, 2017. doi: 10.17487/RFC8174.

[3] D. M. Berry, E. Kamsties, and M. M. Krieger, "From contract drafting to software specification: Linguistic sources of ambiguity," University of Waterloo, Technical Report, 2003.

[4] H. Yang, A. Willis, A. De Roeck, and B. Nuseibeh, "A methodology for automatic identification of nocuous ambiguity," in *Proceedings of COLING*, 2010, pp. 1218–1226.

[5] H. Yang, A. De Roeck, V. Gervasi, A. Willis, and B. Nuseibeh, "Analysing anaphoric ambiguity in natural language requirements," *Requirements Engineering*, vol. 16, pp. 163–189, 2011.

[6] S. Ezzini, S. Abualhaija, C. Arora, and M. Sabetzadeh, "Automated handling of anaphoric ambiguity in requirements: A multi-solution study," in *Proceedings of ICSE*, 2022, pp. 187–199.

[7] A. Veizaga, S. Y. Shin, and L. C. Briand, "Automated smell detection and recommendation in natural language requirements," *IEEE Transactions on Software Engineering*, vol. 50, no. 4, pp. 695–720, 2024. doi: 10.1109/TSE.2024.3361033.

[8] R. Izhar, S. N. Bhatti, and S. A. Alharthi, "Bridging precision and complexity: A novel machine learning approach for ambiguity detection in software requirements," *IEEE Access*, vol. 13, pp. 12014–12031, 2025. doi: 10.1109/ACCESS.2025.3529943.

[9] S. Bashir et al., "Requirements ambiguity detection and explanation with LLMs: An industrial study," in *Proceedings of ICSME*, 2025, pp. 620–631.

[10] L. Zhao et al., "Natural language processing for requirements engineering: A systematic mapping study," *ACM Computing Surveys*, vol. 54, no. 3, pp. 1–41, 2021.

[11] J. Yen, T. Lévai, Q. Ye, X. Ren, R. Govindan, and B. Raghavan, "Semi-automated protocol disambiguation and code generation," in *Proceedings of ACM SIGCOMM*, 2021, pp. 272–286.

[12] M. Pawagi, L. Shao, H. Lee, Y. Sun, and W. Wang, "RFCScope: Detecting logical ambiguities in Internet protocol specifications," in *Proceedings of ASE*, 2025, pp. 1234–1246. doi: 10.1109/ASE63991.2025.00106.

[13] P. Sharma and V. Yegneswaran, "PROSPER: Extracting protocol specifications using large language models," in *Proceedings of HotNets*, 2023. doi: 10.1145/3626111.3628205.

[14] J. Long, J. Su, and B. Han, "LLMs unleashed: Generating protocol code from RFC specifications," in *Proceedings of AAAI*, 2026.

[15] J. Chen, F. Li, Q. Chen, P. Li, L. Xu, and W. Huo, "EBugDec: Detecting inconsistency bugs caused by RFC evolution in protocol implementations," in *Proceedings of RAID*, 2023. doi: 10.1145/3607199.3607222.

[16] M. Zheng, C. Wang, X. Liu, J. Guo, S. Feng, and X. Zhang, "RFCAUDIT: AI agent for auditing protocol implementations against RFC specifications," arXiv:2506.00714v2, 2025.

[17] S. McQuistin et al., "Characterising the IETF through the lens of RFC deployment," in *Proceedings of IMC*, 2021. doi: 10.1145/3487552.3487821.

[18] J. Bian and M. Welzl, "Empowering IETF collaboration with NLP search innovations and LLM-enhanced RFC writing," in *Proceedings of ANRW*, 2025. doi: 10.1145/3744200.3744761.

[19] C. Jiang, W. Xu, and S. Stevens, "ARXIVEDITS: Understanding the Human Revision Process in Scientific Writing," in *Proceedings of EMNLP*, 2022, pp. 9420–9435.

[20] L. Jourdan, F. Boudin, N. Hernandez, and R. Dufour, "CASIMIR: A Corpus of Scientific Articles Enhanced with Multiple Author-Integrated Revisions," in *Proceedings of LREC-COLING*, 2024, pp. 2883–2892.